

Università degli Studi di Padova

Corso di laurea triennale in Ingegneria informatica

Appunti ed esercitazioni di laboratorio

Progettazione di Sistemi Concorrenti

Docente

Prof.ssa Ada Lovelace

Anno accademico

2026–2027

Anno di corso

1

Semestre

1

Autore

Simone Siega

Data

3 agosto 2026

Versione

1.0.0

Indice dei contenuti

1	Modello di un buffer limitato	1
2	Architettura e misure sperimentali	2
3	Scheduling e verifica	3
	Elenchi del documento	4
A	Lista di controllo per il laboratorio	5

1 Modello di un buffer limitato

Un buffer limitato separa la velocità dei produttori da quella dei consumatori senza consentire una crescita incontrollata della memoria. Il modello adottato contiene al massimo N elementi, protegge le transizioni di stato con un mutex e usa due semafori contatori per rappresentare le posizioni occupate e quelle libere. La stessa struttura compare nelle code di rete, nei driver e nelle pipeline di elaborazione.

Definizione 1.1. Lo stato del buffer è la tripla (h, t, n) : h indica la prossima posizione da leggere, t la prossima posizione da scrivere e n il numero di elementi presenti. Gli indici sono calcolati modulo la capacità N .

Teorema 1.1. *Se il contatore parte da $n = 0$ ed è modificato soltanto mentre il mutex è acquisito, ogni esecuzione conserva l'invariante*

$$0 \leq n \leq N.$$

Dimostrazione. Un inserimento può iniziare solo dopo aver riservato una posizione libera; vale quindi $n < N$ e l'aggiornamento produce $n + 1 \leq N$. Una rimozione richiede invece un elemento disponibile, parte da $n > 0$ e produce $n - 1 \geq 0$. Il mutex ordina totalmente gli aggiornamenti e permette di concludere per induzione sul numero di operazioni. $\square$

Sulla macchina di laboratorio la sezione critica non contesa richiede $1,5 \mu\text{s}$; tre misure del risveglio di un thread sono $8,2$, $8,7$ e $9,1 \mu\text{s}$. Conviene quindi preparare i dati fuori dal mutex e limitare la regione protetta all'aggiornamento degli indici e del contatore.

Teorema 1.2. *Se l'inserimento scrive nella posizione t prima di incrementarla e la rimozione legge la posizione h prima di avanzare, gli elementi sono restituiti in ordine FIFO.*

Dimostrazione. Entrambi gli indici avanzano di una posizione modulo N . Una cella non può essere riutilizzata prima della sua lettura, perché il [teorema 1.1](#) impedisce al produttore di superare il consumatore. Le rimozioni visitano pertanto le celle nello stesso ordine in cui sono state riempite. $\square$

Le proprietà di sicurezza non garantiscono da sole il progresso. Uno scheduler non equo può rimandare indefinitamente un thread pronto; la latenza reale dipende quindi anche dall'implementazione dei semafori e dalle politiche del sistema operativo.

2 Architettura e misure sperimentali

La [figura 2.1](#) riassume il percorso dei dati. I produttori completano il parsing prima di entrare nella sezione critica; i consumatori liberano la posizione occupata prima di elaborare il messaggio. Il buffer assume la proprietà dell'elemento al termine dell'inserimento e la trasferisce nuovamente alla rimozione.

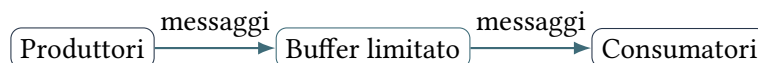


Figura 2.1. Trasferimento della proprietà attraverso il buffer

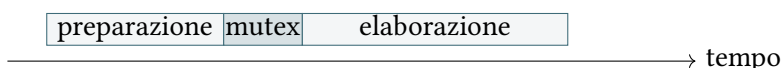


Figura 2.2. Collocazione della sezione critica in una transazione

Il benchmark elabora un milione di messaggi dopo una fase di riscaldamento. Ogni risultato è la mediana di sette esecuzioni e il throughput conta le rimozioni completate, non i tentativi di inserimento. La [tabella 2.1](#) evidenzia il compromesso tra capacità utile e costo della sincronizzazione.

Tabella 2.1. Prestazioni al variare della capacità del buffer

Capacità	Throughput (messaggi/s)	Latenza al 99° percentile
1	79000	44 μ s
16	236000	20 μ s
64	251000	21 μ s
256	249000	29 μ s

Tabella 2.2. Transizioni di stato delle operazioni

Operazione	Precondizione	Modifica atomica dello stato
Inserimento	$n < N$	$a[t] \leftarrow x, t \leftarrow (t + 1) \bmod N, n \leftarrow n + 1$
Rimozione	$n > 0$	$x \leftarrow a[h], h \leftarrow (h + 1) \bmod N, n \leftarrow n - 1$
Ispezione	Mutex acquisito	Lettura di (h, t, n) senza modifiche

```

1 def enqueue(queue, item):
2     with queue.mutex:
3         if queue.count == queue.capacity:
4             raise BufferError("buffer pieno")
5         queue.data[queue.tail] = item
6         queue.tail = (queue.tail + 1) % queue.capacity
7         queue.count += 1
  
```

Listato 2.1. Inserimento con controllo esplicito

3 Scheduling e verifica

Un inserimento bloccante riserva prima una posizione libera e acquisisce poi il mutex. Invertire le due operazioni può causare un deadlock: il produttore conserverebbe il mutex in attesa di un consumatore che necessita dello stesso mutex. La rimozione usata nei test è riportata nel [listato 3.1](#).

```

1 def dequeue(queue):
2     with queue.mutex:
3         if queue.count == 0:
4             raise BufferError("buffer vuoto")
5         item = queue.data[queue.head]
6         queue.head = (queue.head + 1) % queue.capacity
7         queue.count -= 1
8     return item

```

Listato 3.1. Rimozione con controllo esplicito

I controlli nei [listati 2.1](#) e [3.1](#) trasformano una violazione dell'invariante in un errore immediato. Il codice di produzione attende normalmente un semaforo, ma le asserzioni sul contatore restano utili per individuare segnali errati e accessi eseguiti senza lock. L'[algoritmo 3.1](#) adotta l'ordine di acquisizione sicuro.

Dati: Una coda q e un elemento x
Risultato: La coda aggiornata

```

1 attendi( $q.libere$ )
2 acquisisci( $q.mutex$ )
3  $q.dati[q.coda] \leftarrow x$ 
4  $q.coda \leftarrow (q.coda + 1) \bmod q.capacita$ 
5 rilascia( $q.mutex$ )
6 segnala( $q.usate$ )
7 restituisci  $q$ 

```

Algoritmo 3.1. Inserimento bloccante

La procedura nell'[algoritmo 3.2](#) non modifica lo stato. Può essere eseguita dopo ogni operazione casuale di un test basato su modello, confrontando la coda reale con una semplice sequenza di riferimento.

Dati: Lo stato di una coda q
Risultato: Vero se e solo se lo stato è valido

```

1 se  $q.conteggio < 0$  o  $q.conteggio > q.capacita$  allora
2     restituisci falso
3 fine
4 per  $i \leftarrow 1$  a  $q.conteggio$  fai
5     controlla l'elemento in  $(q.testa + i - 1) \bmod q.capacita$ 
6 fine
7 restituisci vero

```

Algoritmo 3.2. Validazione dello stato della coda

I riferimenti singolari minuscoli sono [teorema 1.1](#), [figura 2.1](#), [tabella 2.1](#), [listato 2.1](#) e [algoritmo 3.1](#). Le forme singolari maiuscole sono [Teorema 1.1](#), [Figura 2.1](#), [Tabella 2.1](#), [Listato 2.1](#) e [Algoritmo 3.1](#).

Le forme plurali minuscole sono [teoremi 1.1](#) e [1.2](#), [figure 2.1](#) e [2.2](#), [tabelle 2.1](#) e [2.2](#), [listati 2.1](#) e [3.1](#) e [algoritmi 3.1](#) e [3.2](#). Le forme plurali maiuscole sono [Teoremi 1.1](#) e [1.2](#), [Figure 2.1](#) e [2.2](#), [Tabelle 2.1](#) e [2.2](#), [Listati 2.1](#) e [3.1](#) e [Algoritmi 3.1](#) e [3.2](#).

Elenchi del documento

Elenco delle figure

2.1	Trasferimento della proprietà attraverso il buffer	2
2.2	Collocazione della sezione critica in una transazione	2

Elenco delle tabelle

2.1	Prestazioni al variare della capacità del buffer	2
2.2	Transizioni di stato delle operazioni	2

Elenco degli algoritmi

3.1	Inserimento bloccante	3
3.2	Validazione dello stato della coda	3

Elenco dei listati

2.1	Inserimento con controllo esplicito	2
3.1	Rimozione con controllo esplicito	3

A Lista di controllo per il laboratorio

Prima delle misure, compilare il programma con le ottimizzazioni attive ma senza eliminare le asserzioni. È opportuno fissare l'affinità dei thread quando si confrontano strategie diverse, annotare il modello del processore e la versione del sistema operativo, e mantenere invariato il carico di ingresso. Una fase iniziale non misurata stabilizza cache e frequenza della CPU.

Dopo ogni operazione si deve verificare $0 \leq n \leq N$ e confrontare la sequenza rimossa con il modello di riferimento. Le capacità 1, 2 e un valore non potenza di due sono casi particolarmente efficaci: il primo espone errori di ordinamento, mentre gli altri sollecitano il riavvolgimento degli indici.

La relazione finale riporta revisione del sorgente, opzioni del compilatore, numero di ripetizioni, metodo di aggregazione e osservazioni grezze. Throughput e latenza devono rimanere distinti; gli outlier vanno spiegati, non eliminati senza motivazione. In questo modo l'esperimento resta riproducibile e le conclusioni sono legate all'implementazione effettivamente misurata.